balance flexibility and complexity, so that we only need to add a single real number to the model of each edge, the distance of the control point from the segment between the endpoints (denoted with `d` in the figure). It's also advisable (but not strictly necessary) to restrict the possible values of `d` so that its absolute value will be in the order of magnitude of `w`, the distance between the endpoints; negative values of `d` will cause the convexity of the edge to flip (in figure 16.6, with a negative value for `d`, the curve would be drawn below the segment `uv` passing through the vertices).

16.2 How Optimization Works

So, random sampling seems to works: if you try the version implemented in *JsGraphs* library, you can see that (on average) the crossing number provided by the random algorithm with restart is better than the one returned by the one-pass random version. Intuitively, we can understand why: if we toss a coin 100 times it's easier to get at least one head than if we tossed the same coin just once.

To see, in a more formal framework, how this operates, we could use a visualization of how the optimization proceeds. This is not trivial, because what we are optimizing here is a function of `2n` parameters, when a graph has `n` vertices: for each vertex, we can change its `x` and `y` coordinates.

Now, humans are really good at visualizing functions of 1 parameter in a 2D plot: the `x` axis is usually the parameter, and the `y` axis shows the value of the function; with 2 parameters, we can still visualize things meaningfully: until AR/VR goes mainstream, we have to make do with a 2D projection of a 3D plot, which is not ideal, but still feasible (as you'll see in figure 16.8).